Khoa học Dữ liệu

Notebook: Xử lý và trực quan hóa dữ liệu một chiều
Năm học: 2025-2026

---

1. Xử lý, tính toán giá trị tổng hợp với dữ liệu

In [43]:
```
# Import các thư viện cần thiết
from google.colab import drive
import pandas as pd
import numpy as np

drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/driv
e", force_remount=True).
```

Việc xử lý và tính toán các giá trị tổng hợp là bước quan trọng trong quá trình phân tích dữ liệu. Giai đoạn này giúp người phân tích hiểu rõ đặc điểm, xu hướng và sự phân bố của dữ liệu, từ đó phát hiện các bất thường hoặc giá trị ngoại lai. Đồng thời, quá trình tiền xử lý còn giúp chuyển đổi dữ liệu về dạng chuẩn, đảm bảo tính nhất quán và hỗ trợ cho các bước tính toán, mô hình hóa sau này được thực hiện hiệu quả và chính xác hơn.

Ở đây, chúng ta sẽ thực hiện trên dữ liệu `catchment.csv` về lưu vực/hydrogeological catchments trong bối cảnh địa chất – thủy văn được lưu trư ở định dạng csv với các biến:

- lithology: Thành phần thạch học (loại đá nền trong lưu vực)
- age: Niên đại địa chất của đá nền
- springs: Số lượng suối (springs) trong catchment đó.
- pH: Độ axit/kiềm của nước.
- CaMg: Hàm lượng Ca + Mg (ion kiềm thổ) trong nước.
- vegetation: Độ phủ thực vật (%).

In [44]:
```
url_catchments = '/content/drive/MyDrive/course_Data_Science_for_Geoscience/Week 1/Dataset/catchments.c
catchments = pd.read_csv(url_catchments)
```

In [45]:
```
catchments.head(5)  #In ra 5 dòng đầu tiên của dữ liệu
```

Out[45]:

	lithology	age	springs	pH	CaMg	vegetation
0	basalt	Cenozoic	0	6.2	0.35	5.8
1	granite	Mesozoic	1	4.4	11.00	28.0
2	basalt	Mesozoic	0	5.6	6.00	12.0
3	sandstone	Cenozoic	6	5.2	1.80	27.0
4	shale	Precambrian	3	4.5	2.30	40.0

`df.info()` là một phương thức trong thư viện Pandas của Python, cung cấp một bản tóm tắt ngắn gọn về một DataFrame. Đây là một công cụ quan trọng để khám phá dữ liệu ban đầu và hiểu cấu trúc cũng như nội dung của

tập dữ liệu.

Khi bạn dùng `df.info()`, nó sẽ in thông tin ra bảng điều khiển, bao gồm:

- Index dtype và range: Thông tin về chỉ mục của DataFrame, chẳng hạn như RangeIndex và số lượng mục nhập.
- Tên cột và kiểu dữ liệu: Danh sách tất cả các cột trong DataFrame và kiểu dữ liệu tương ứng của chúng (ví dụ: int64, float64, object, datetime64[ns]).
- Giá trị không null: Số lượng mục nhập không null cho mỗi cột, giúp xác định dữ liệu bị thiếu.

In [46]:
```
catchments.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   lithology   20 non-null     object
 1   age         20 non-null     object
 2   springs     20 non-null     int64
 3   pH          20 non-null     float64
 4   CaMg        20 non-null     float64
 5   vegetation  20 non-null     float64
dtypes: float64(3), int64(1), object(2)
memory usage: 1.1+ KB
```

Để trích xuất các cột chứa các thông tin cần thiết có thể dùng:

In [47]:
```
catchments[['pH', 'CaMg', 'vegetation']]
```

Out[47]:

	pH	CaMg	vegetation
0	6.2	0.35	5.8
1	4.4	11.00	28.0
2	5.6	6.00	12.0
3	5.2	1.80	27.0
4	4.5	2.30	40.0
5	5.4	0.59	12.0
6	4.8	8.40	3.8
7	5.9	2.90	6.3
8	3.8	5.90	17.0
9	4.0	2.10	16.0
10	5.0	1.20	95.0
11	4.1	2.10	94.0
12	5.2	1.10	92.0
13	5.5	1.60	88.0
14	5.3	0.90	88.0
15	4.6	1.70	70.0
16	5.7	3.40	92.0
17	4.6	0.53	72.0
18	4.6	2.20	74.0
19	5.6	7.70	84.0

Tương tự như thực hiện tính toán trên mảng, với dữ liệu dạng **DataFrame** cũng sẽ có một số hàm hỗ trợ cho việc tính toán như `count()`, `len()`, `mean()`, `std()`, `quantile()`, `...`

In [48]:
```
catchments['pH'].mean()
```

Out[48]:
```
np.float64(4.999999999999999)
```

In [49]:
```
catchments['pH'].std(ddof=1)
```

Out[49]:
```
0.6657010550816652
```

In [50]:
```
np.quantile(catchments['pH'], 0.25)
```

Out[50]:
```
np.float64(4.574999999999999)
```

Với dữ liệu có nhiều cột và nhiều dòng, việc quan sát toàn bộ dữ liệu rất khó khăn trong việc tập trung vào thông tin nhất định. Vì vậy, việc lọc ra dữ liệu theo một hoặc nhiều tiêu chí giúp tập trung phân tích đúng đối tượng cần quan tâm thay vì làm việc với toàn bộ dữ liệu.

Như ví dụ ở phía dưới, việc có nhiều loại đá nền `lithology` khiến nhiễu loạn giữa các thông tin, chúng ta có thể lọc ra nhóm đá nền nhất định và phân tích chúng.

In [51]:
```
catchments[catchments['lithology'] == 'basalt']
```

Out[51]:

	lithology	age	springs	pH	CaMg	vegetation
0	basalt	Cenozoic	0	6.2	0.35	5.8
2	basalt	Mesozoic	0	5.6	6.00	12.0
5	basalt	Mesozoic	1	5.4	0.59	12.0
7	basalt	Palaeozoic	0	5.9	2.90	6.3
13	basalt	Cenozoic	1	5.5	1.60	88.0
16	basalt	Mesozoic	0	5.7	3.40	92.0
19	basalt	Cenozoic	1	5.6	7.70	84.0

Ngoài việc lọc dữ liệu dựa trên các thông tin định tính, việc lọc dữ liệu với các điều kiện trên các biến định lượng cũng có thể loại bỏ đi một vài giá trị không cần thiết

In [52]:
```
catchments[catchments['pH'] > 5]
```

Out[52]:

	lithology	age	springs	pH	CaMg	vegetation
0	basalt	Cenozoic	0	6.2	0.35	5.8
2	basalt	Mesozoic	0	5.6	6.00	12.0
3	sandstone	Cenozoic	6	5.2	1.80	27.0
5	basalt	Mesozoic	1	5.4	0.59	12.0
7	basalt	Palaeozoic	0	5.9	2.90	6.3
12	sandstone	Mesozoic	4	5.2	1.10	92.0
13	basalt	Cenozoic	1	5.5	1.60	88.0
14	sandstone	Palaeozoic	4	5.3	0.90	88.0
16	basalt	Mesozoic	0	5.7	3.40	92.0
19	basalt	Cenozoic	1	5.6	7.70	84.0

Và việc kết hợp nhiều điều kiện để lọc dữ liệu cũng có thể hoạt động với dữ liệu dạng bảng như: `df[(điều kiện 1) & (điều kiện 2)]` như ví dụ phía dưới:

In [53]:
```
catchments[(catchments['pH'] > 5) & (catchments['CaMg'] > 2)]
```

Out[53]:

	lithology	age	springs	pH	CaMg	vegetation
2	basalt	Mesozoic	0	5.6	6.0	12.0
7	basalt	Palaeozoic	0	5.9	2.9	6.3
16	basalt	Mesozoic	0	5.7	3.4	92.0
19	basalt	Cenozoic	1	5.6	7.7	84.0

Bên cạnh việc lọc dữ liệu để chọn ra các dòng thỏa điều kiện cụ thể, thì sắp xếp dữ liệu cũng là một bước quan trọng trong quá trình phân tích và khám phá dữ liệu.

- Khi sắp xếp dữ liệu theo một cột, ví dụ như pH, ta có thể dễ dàng nhận ra các giá trị quá thấp hoặc quá cao, từ đó phát hiện các điểm ngoại lai (outliers) hoặc xu hướng bất thường trong mẫu dữ liệu.

Ví dụ: sắp xếp theo pH tăng dần giúp phát hiện ngay các mẫu có độ axit cao (pH thấp), trong khi sắp xếp giảm dần giúp phát hiện các mẫu có tính kiềm mạnh (pH cao).

- Hỗ trợ việc quan sát và trình bày dữ liệu. Khi dữ liệu được sắp xếp có trật tự (ví dụ theo zinc, lead, hoặc elev), việc đọc, đối chiếu, và so sánh các giá trị trở nên rõ ràng và trực quan hơn.
- Dữ liệu có trật tự cũng giúp trực quan hóa (vẽ biểu đồ) trở nên chính xác và dễ hiểu hơn.
- Tạo thuận lợi cho các bước xử lý hoặc phân tích tiếp theo. Một số phép phân tích thống kê hoặc mô hình học máy yêu cầu dữ liệu đầu vào phải được sắp xếp hoặc chia nhóm dữ liệu.
- Việc sắp xếp cho phép nhận thấy mối tương quan giữa các biến (chẳng hạn, khi elev tăng thì zinc có giảm không?) hoặc phát hiện quy luật tiềm ẩn trong bộ dữ liệu.

In [54]:
```
catchments.sort_values(['pH'])
```

Out[54]:

	lithology	age	springs	pH	CaMg	vegetation
8	shale	Mesozoic	1	3.8	5.90	17.0
9	shale	Mesozoic	2	4.0	2.10	16.0
11	shale	Precambrian	2	4.1	2.10	94.0
1	granite	Mesozoic	1	4.4	11.00	28.0
4	shale	Precambrian	3	4.5	2.30	40.0
15	granite	Palaeozoic	0	4.6	1.70	70.0
18	granite	Mesozoic	0	4.6	2.20	74.0
17	shale	Palaeozoic	1	4.6	0.53	72.0
6	shale	Palaeozoic	2	4.8	8.40	3.8
10	sandstone	Palaeozoic	4	5.0	1.20	95.0
3	sandstone	Cenozoic	6	5.2	1.80	27.0
12	sandstone	Mesozoic	4	5.2	1.10	92.0
14	sandstone	Palaeozoic	4	5.3	0.90	88.0
5	basalt	Mesozoic	1	5.4	0.59	12.0
13	basalt	Cenozoic	1	5.5	1.60	88.0
2	basalt	Mesozoic	0	5.6	6.00	12.0
19	basalt	Cenozoic	1	5.6	7.70	84.0
16	basalt	Mesozoic	0	5.7	3.40	92.0
7	basalt	Palaeozoic	0	5.9	2.90	6.3
0	basalt	Cenozoic	0	6.2	0.35	5.8

Với hàm `DataFrame.sort_values([cột cần sắp xếp])` mặc định sẽ sắp xếp tăng dần, để sắp xếp giảm dần thì cần thêm tham số `ascending = False` như ví dụ:

In [55]:
```
catchments.sort_values(['pH'], ascending=False)
```

Out[55]:

	lithology	age	springs	pH	CaMg	vegetation
0	basalt	Cenozoic	0	6.2	0.35	5.8
7	basalt	Palaeozoic	0	5.9	2.90	6.3
16	basalt	Mesozoic	0	5.7	3.40	92.0
2	basalt	Mesozoic	0	5.6	6.00	12.0
19	basalt	Cenozoic	1	5.6	7.70	84.0
13	basalt	Cenozoic	1	5.5	1.60	88.0
5	basalt	Mesozoic	1	5.4	0.59	12.0
14	sandstone	Palaeozoic	4	5.3	0.90	88.0
12	sandstone	Mesozoic	4	5.2	1.10	92.0
3	sandstone	Cenozoic	6	5.2	1.80	27.0
10	sandstone	Palaeozoic	4	5.0	1.20	95.0
6	shale	Palaeozoic	2	4.8	8.40	3.8
17	shale	Palaeozoic	1	4.6	0.53	72.0
15	granite	Palaeozoic	0	4.6	1.70	70.0
18	granite	Mesozoic	0	4.6	2.20	74.0
4	shale	Precambrian	3	4.5	2.30	40.0
1	granite	Mesozoic	1	4.4	11.00	28.0
11	shale	Precambrian	2	4.1	2.10	94.0
9	shale	Mesozoic	2	4.0	2.10	16.0
8	shale	Mesozoic	1	3.8	5.90	17.0

Chúng ta cũng có thể sắp xếp dữ liệu theo nhiều tiêu chí cột khác nhau và cách sắp xếp khác nhau. Ví dụ như sắp xếp dữ liệu `pH` giảm dần và các giá trị `lithology` lại tăng dần

In [56]:
```
catchments.sort_values(['pH', 'lithology'], ascending=[False, True])
```

Out[56]:

	lithology	age	springs	pH	CaMg	vegetation
0	basalt	Cenozoic	0	6.2	0.35	5.8
7	basalt	Palaeozoic	0	5.9	2.90	6.3
16	basalt	Mesozoic	0	5.7	3.40	92.0
2	basalt	Mesozoic	0	5.6	6.00	12.0
19	basalt	Cenozoic	1	5.6	7.70	84.0
13	basalt	Cenozoic	1	5.5	1.60	88.0
5	basalt	Mesozoic	1	5.4	0.59	12.0
14	sandstone	Palaeozoic	4	5.3	0.90	88.0
3	sandstone	Cenozoic	6	5.2	1.80	27.0
12	sandstone	Mesozoic	4	5.2	1.10	92.0
10	sandstone	Palaeozoic	4	5.0	1.20	95.0
6	shale	Palaeozoic	2	4.8	8.40	3.8
15	granite	Palaeozoic	0	4.6	1.70	70.0
18	granite	Mesozoic	0	4.6	2.20	74.0
17	shale	Palaeozoic	1	4.6	0.53	72.0
4	shale	Precambrian	3	4.5	2.30	40.0
1	granite	Mesozoic	1	4.4	11.00	28.0
11	shale	Precambrian	2	4.1	2.10	94.0
9	shale	Mesozoic	2	4.0	2.10	16.0
8	shale	Mesozoic	1	3.8	5.90	17.0

Ngoài việc lọc (filter) và sắp xếp (sort) dữ liệu theo một hoặc nhiều tiêu chí, thì việc nhóm dữ liệu (grouping) theo các biến định tính (categorical variables) cũng là một kỹ thuật quan trọng trong quá trình phân tích dữ liệu.

Cụ thể, phương pháp này thường được thực hiện bằng hàm `groupby()` trong Pandas, cho phép ta chia dữ liệu thành các nhóm nhỏ hơn dựa trên giá trị của một hay nhiều cột định tính.

In [57]:
```
df = catchments.groupby(['lithology','age'])
df.get_group(('basalt', 'Cenozoic'))
```

Out[57]:

	lithology	age	springs	pH	CaMg	vegetation
0	basalt	Cenozoic	0	6.2	0.35	5.8
13	basalt	Cenozoic	1	5.5	1.60	88.0
19	basalt	Cenozoic	1	5.6	7.70	84.0

Sau khi nhóm dữ liệu, ta có thể dễ dàng sử dụng các hàm như trung bình (mean), trung vị (median), phương sai (var), hoặc tổng (sum) cho từng nhóm.

Ví dụ: nhóm theo cột lithology để xem mức trung bình của pH giữa các loại đá khác nhau có chênh lệch đáng kể hay không.

In [58]:
```
catchments.groupby(['lithology']).count()
```

Out[58]:

lithology	age	springs	pH	CaMg	vegetation
basalt	7	7	7	7	7
granite	3	3	3	3	3
sandstone	4	4	4	4	4
shale	6	6	6	6	6

In [59]:
```
catchments.groupby('lithology')['pH'].mean()
```

Out[59]:

lithology	pH
basalt	5.700000
granite	4.533333
sandstone	5.175000
shale	4.300000

dtype: float64

2. Trực quan hóa dữ liệu một chiều

Trực quan hóa dữ liệu một chiều là quá trình biểu diễn đồ họa của một biến duy nhất — có thể là biến định lượng hoặc biến định tính — nhằm giúp hiểu rõ hơn về đặc điểm, phân bố và xu hướng của biến đó trong tập dữ liệu.

In [60]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

Một số loại biểu đồ và một số hàm với `matplotlib`:

- Đường (line plot) → plt.plot(x, y)
- Phân tán (scatter plot) → plt.scatter(x, y)
- Cột (bar chart) → plt.bar(x, y)
- Phân bố (histogram) → plt.hist(x)
- Hộp (boxplot) → plt.boxplot(x)

Thêm nhãn và tiêu đề trong matplotlib

```
plt.title("Tên tiêu đề")
plt.xlabel("Tên trục x")
plt.ylabel("Tên trục y")
```

Để vẽ biểu đồ luôn cần có: `plt.show()`

Một số tùy chỉnh hiển thị (nếu cần):

- Thêm lưới (grid):

`plt.grid(True, linestyle='--', alpha=0.6)`

- Giới hạn trục:

```
plt.xlim(0, 14)
plt.ylim(0, 10)
```

2.1 Histogram

Histogram là một trong những công cụ trực quan hóa dữ liệu quan trọng nhất trong phân tích thống kê. Nó giúp ta quan sát phân bố tần suất (frequency distribution) của một biến định lượng (numerical variable).

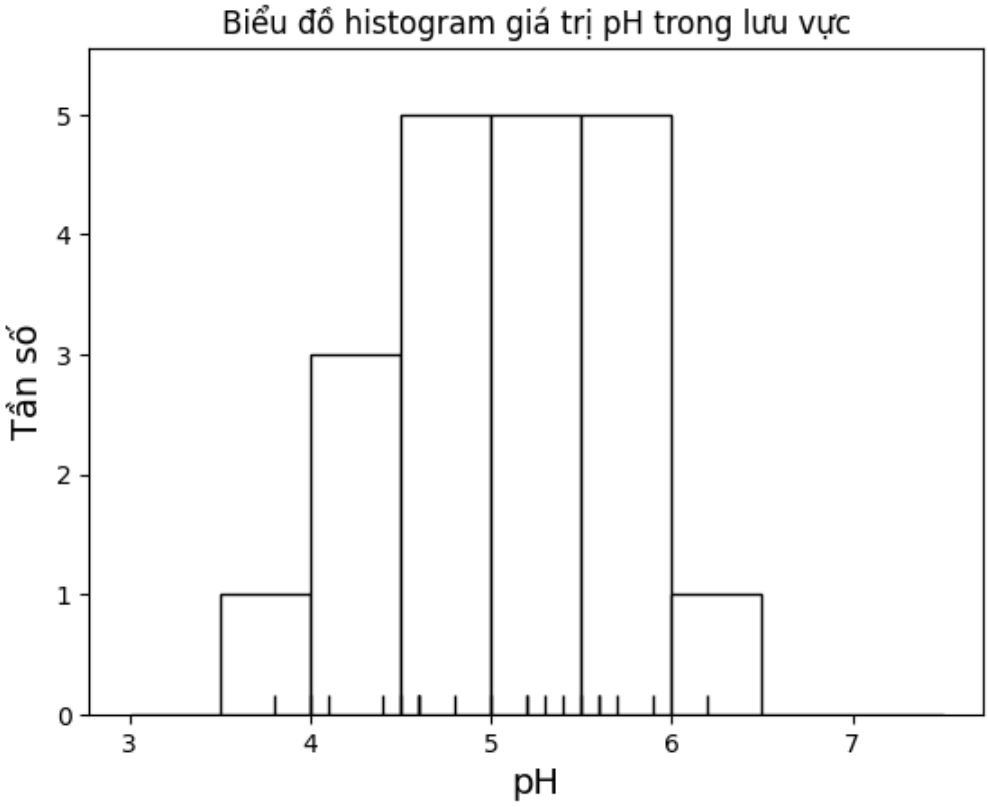
Histogram được dùng để:

- Xem dữ liệu phân bố như thế nào — có lệch trái, lệch phải hay đối xứng.
- Xác định độ tập trung (central tendency) và độ phân tán (spread) của dữ liệu.
- Phát hiện giá trị ngoại lai (outliers) hoặc các mô hình đặc biệt trong dữ liệu.

Một số tham số thường dùng trong `plt.hist()`:

- `bins`: Số lượng khoảng (cột) trên trục x
- `color`: Màu cột histogram
- `edgecolor`: Màu viền của các cột

In [61]:
```
# Histogram với break
plt.hist(
    catchments['pH'],
    np.arange(3, 8, 0.5),
    color="white",
    edgecolor="black"
)

# Rug plot
sns.rugplot(
    x=catchments['pH'],
    height=0.03,
    color="black"
)

# Nhãn trục
plt.xlabel("pH", fontsize=14)
plt.ylabel("Tần số", fontsize=14)
plt.title("Biểu đồ histogram giá trị pH trong lưu vực")

plt.show()
```



2.2 Biểu đồ hộp

Hàm cho việc vẽ biểu đồ hộp trong `Matplotlib`:

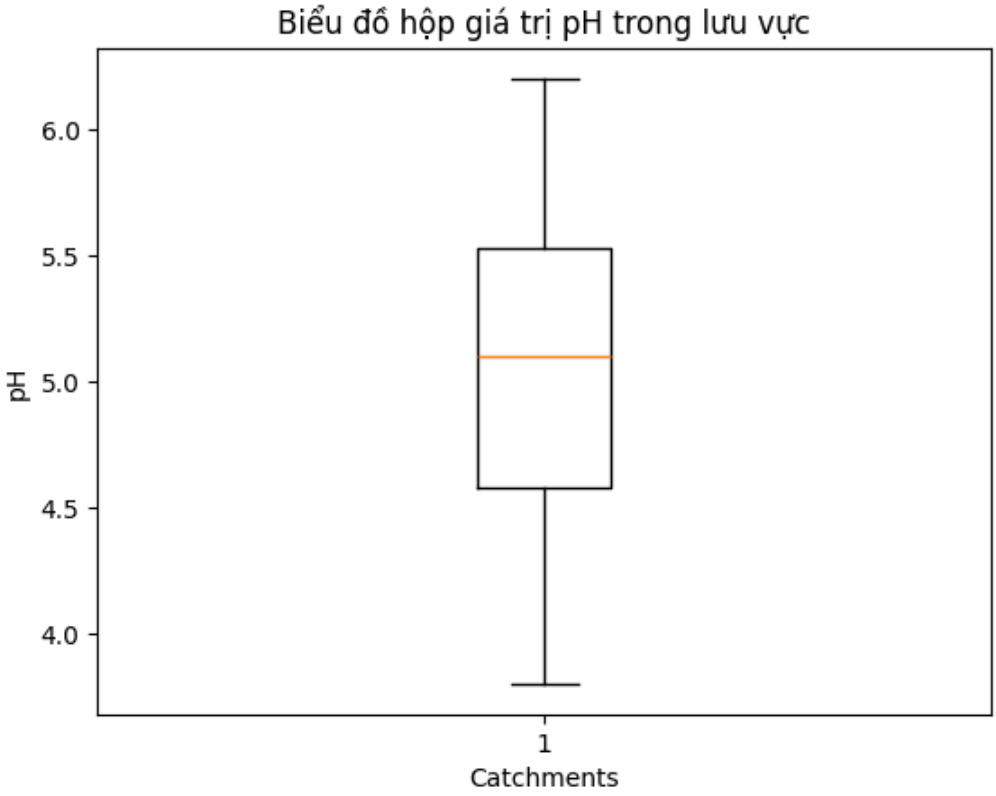
```
plt.boxplot(data)
```

Một số tham số trong hàm `boxplot` từ `Matplotlib`:

- `vert`: Quy định chiều vẽ của boxplot. True → dọc, False → ngang.
- `patch_artist`: Nếu True, hộp sẽ được tô màu (filled boxes).
- `boxprops`: Thuộc tính của hộp (nền & viền).

  ```
  dict(facecolor='lightblue', color='navy', linewidth=2)
  ```

In [62]:
```
plt.boxplot(catchments['pH'])

plt.title("Biểu đồ hộp giá trị pH trong lưu vực")
plt.xlabel("Catchments")
plt.ylabel("pH")

plt.show()
```



2.3 Hàm phân phối tích lũy thực nghiệm (ECDF)

Hàm ECDF cho ta biết, tại mỗi giá trị x:

Có bao nhiêu phần trăm dữ liệu có giá trị ≤ x.

Đường ECDF tăng dần từ 0 đến 1, thể hiện sự tích lũy tần suất của các giá trị trong mẫu.

Đồ thị ECDF thường có dạng bậc thang, vì mỗi giá trị quan sát làm tăng hàm ECDF thêm một bước bằng $1/n$

Nhờ đặc điểm này, ECDF là công cụ mạnh để:

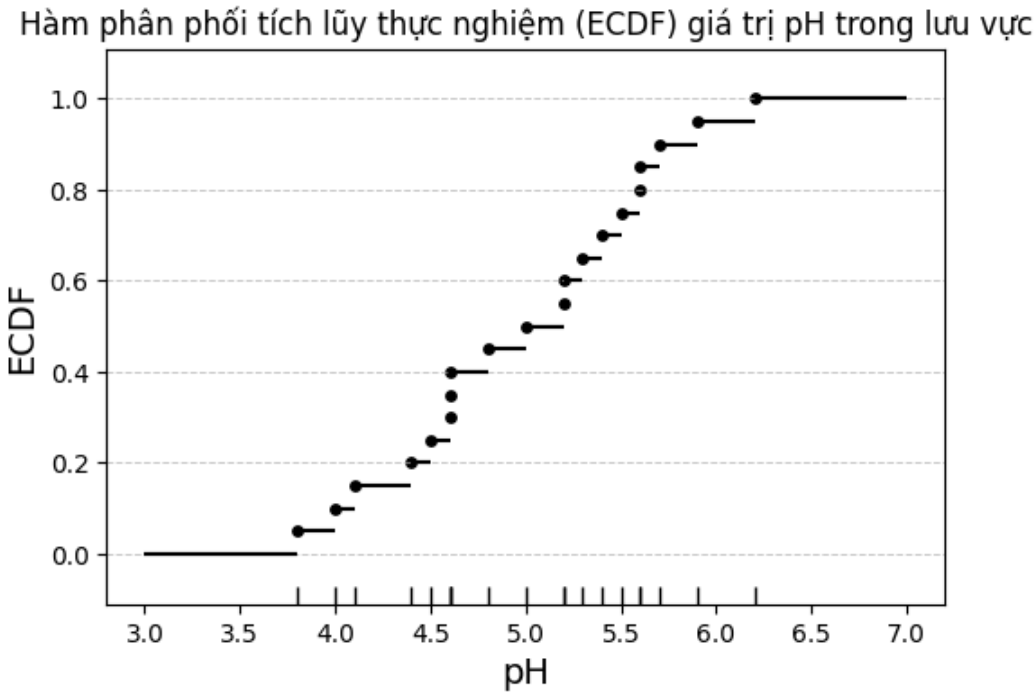
- Mô tả phân phối dữ liệu một cách chính xác và không phụ thuộc vào việc chia nhóm (bins) như trong histogram.
- So sánh nhiều tập dữ liệu (chẳng hạn so sánh giữa các nhóm hoặc các phép đo khác nhau).
- Phát hiện outliers hoặc các khác biệt về xu hướng và độ lệch phân phối.

In [63]:
```
# Tính ECDF
def ecdf_fun(x):
    x = np.sort(pd.Series(x).dropna().values)
    n = len(x)
    y = np.arange(1, n + 1) / n
    return pd.DataFrame({"x": x, "y": y})
```

In [64]:
```
res_ph_ecdf = ecdf_fun(catchments['pH'])
```

In [65]:
```
plt.figure(figsize=(6, 4))

# Vẽ ECDF
plt.scatter(res_ph_ecdf["x"], res_ph_ecdf["y"], color="black", s=15)

x0 = np.concatenate(([3], res_ph_ecdf["x"]))
x1 = np.concatenate((res_ph_ecdf["x"], [7]))
y0 = np.concatenate(([0], res_ph_ecdf["y"]))
```

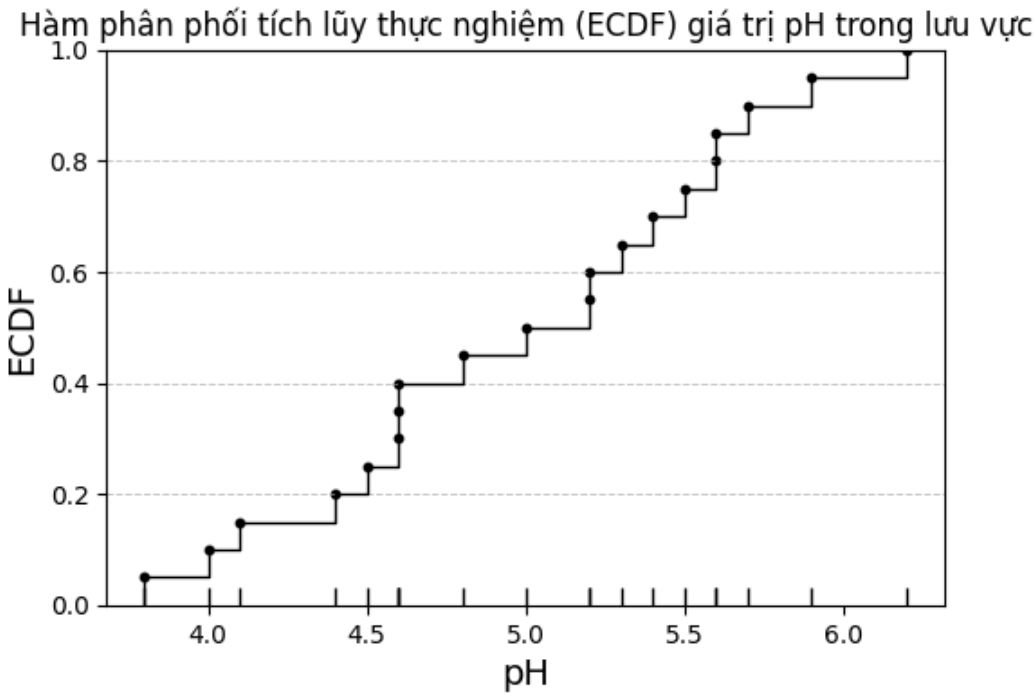
```
plt.hlines(y=y0, xmin=x0, xmax=x1, colors="black")

sns.rugplot(
    x=catchments['pH'],
    height=0.03,
    color="black"
)

# Nhãn trục
plt.xlabel("pH", fontsize=14)
plt.ylabel("ECDF", fontsize=14)
plt.title("Hàm phân phối tích lũy thực nghiệm (ECDF) giá trị pH trong lưu vực")

plt.grid(True, axis="y", linestyle="--", linewidth=0.7, alpha=0.7)

plt.show()
```



Ngoài cách vẽ thủ công như bên trên thì thư viện `Seaborns` cũng có hàm `ecdfplot` tự tính toán và vẽ hình ở dạng bậc thang.

In [66]:
```
plt.figure(figsize=(6, 4))
plt.scatter(res_ph_ecdf["x"], res_ph_ecdf["y"], color="black", s=10)
sns.ecdfplot(
    data=catchments,
    x="pH",
    color="black",
    linewidth=1
)

# Rug plot
sns.rugplot(
    x=catchments["pH"],
    height=0.03,
    color="black"
)

# Nhãn và giới hạn
plt.xlabel("pH", fontsize=14)
plt.ylabel("ECDF", fontsize=14)
plt.title("Hàm phân phối tích lũy thực nghiệm (ECDF) giá trị pH trong lưu vực")

plt.grid(True, axis="y", linestyle="--", linewidth=0.7, alpha=0.7)
```

```
plt.show()
```



2.4 Ước lượng mật độ thực nghiệm (Empirical Density Estimator)

Sau khi có **hàm phân phối tích lũy thực nghiệm (ECDF)** $F_n(y)$, ta nhận thấy rằng hàm này **không liên tục** — tức là giá trị của $F_n(y)$ chỉ tăng lên tại những điểm dữ liệu có trong mẫu.

Tuy nhiên, trong nhiều trường hợp, ta cần **xấp xỉ hàm mật độ xác suất** $f(y)$, tức là **đạo hàm của hàm phân phối** $F(y)$.

Để làm điều đó, ta có thể **ước lượng đạo hàm của ECDF** bằng **xấp xỉ sai phân trung tâm (central difference approximation)** với bước nhảy $h > 0$:

$$\hat{f}_n(y) = \frac{F_n(y+h/2) - F_n(y-h/2)}{h}$$

Công thức này được gọi là **ước lượng mật độ thực nghiệm (Empirical Density Estimator – EDENS)**.
Về bản chất, nó đo **mức độ thay đổi của ECDF quanh điểm** y.

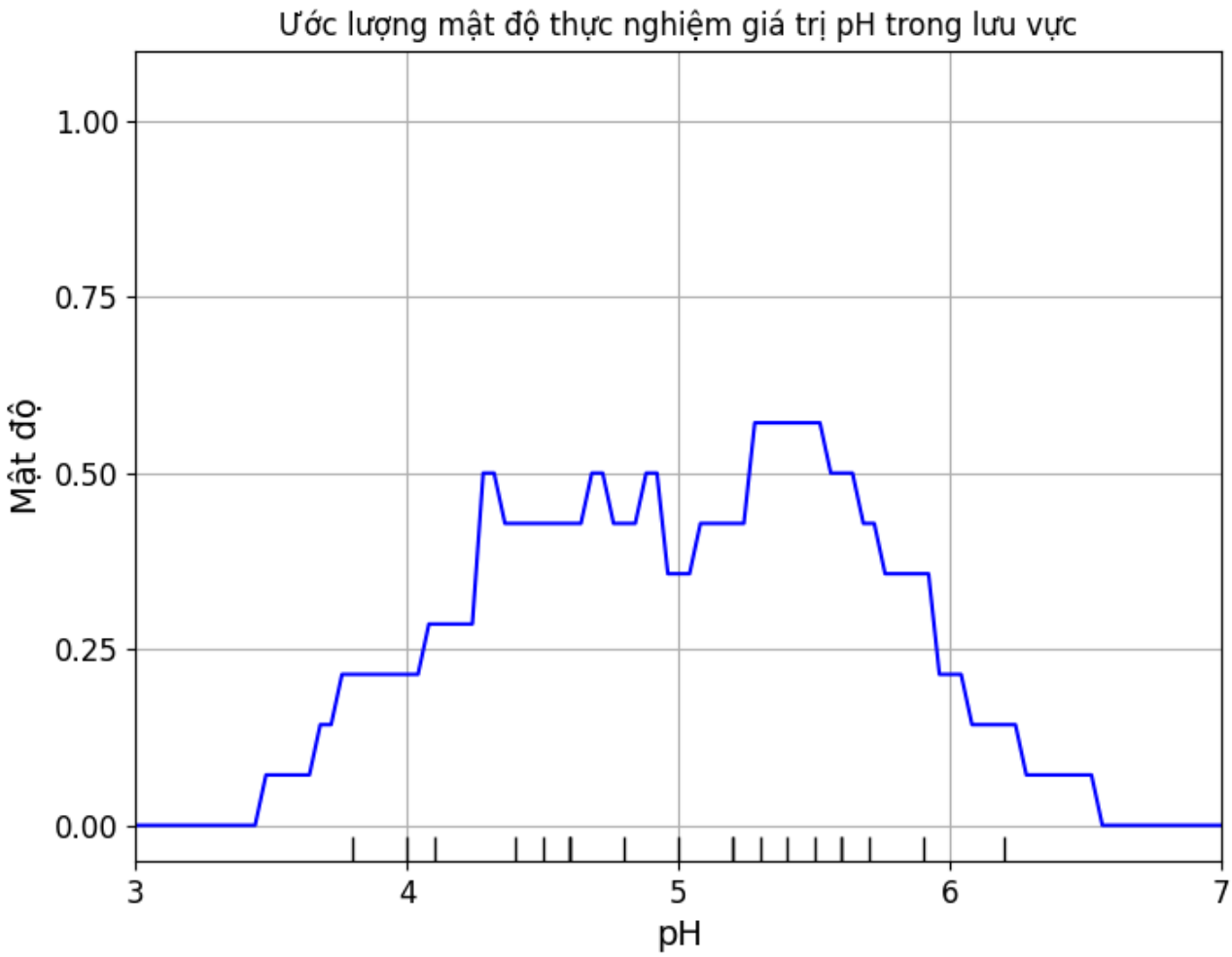
Nếu ta tưởng tượng ECDF như một **đường cong tăng dần**, thì EDENS chính là **độ dốc trung bình của đường cong đó** trong khoảng độ rộng h.

In [67]:
```
def edens_fun(x, x0=None, h=None):
    x = np.sort(pd.Series(x).dropna().values)
    n = len(x)
    if h is None:
        h = 2 * (np.percentile(x, 75) - np.percentile(x, 25)) * n**(-1/3)
    if x0 is None:
        x0 = np.linspace(3, 7, 200)
    y_vals = []
    for y in x0:
        # uniform kernel: điều kiện |x - y|/h <= 0.5
        val = np.mean(np.abs(x - y) / h <= 0.5)
        y_vals.append(val)
    y_vals = np.array(y_vals) / h
    return pd.DataFrame({"x": x0, "y": y_vals})
```

In [68]:
```
iqr = np.quantile(catchments['pH'], 0.75) - np.quantile(catchments['pH'], 0.25)
h_opt = 2 * iqr * len(catchments['pH'])**(-1/3)
```

```
res_ph_edens_025 = edens_fun(catchments['pH'], x0=np.linspace(3, 7, 101))
```

In [69]:
```
# Vẽ hình
plt.figure(figsize=(8,6))
plt.plot(res_ph_edens_025["x"], res_ph_edens_025["y"], color="blue")

# Rug plot
sns.rugplot(
    x=catchments["pH"],
    height=0.03,
    color="black"
)

plt.xlabel("pH", fontsize=14)
plt.ylabel("Mật độ", fontsize=14)
plt.title("Ước lượng mật độ thực nghiệm giá trị pH trong lưu vực")

plt.xticks(np.arange(3, 8, 1), fontsize=12)
plt.yticks(np.arange(0, 1.25, 0.25), fontsize=12)
plt.xlim(3, 7)
plt.ylim(-0.05, 1.1)
plt.grid(True)
plt.show()
```



Một trong những **hạn chế** của phương pháp ước lượng mật độ thực nghiệm (EDENS) là **đường mật độ thu được không trơn**, mà **bị gấp khúc** .
Điều này khiến cho ước lượng ($\hat{f}_n(y)$) **không liên tục**, làm **giảm tính thẩm mỹ của đồ thị** .

Để **khắc phục vấn đề này**, ta có thể bằng một **hàm dương, trơn, liên tục và khả vi**, đồng thời **bảo đảm rằng ước lượng mật độ thu được vẫn thỏa mãn tính chất của hàm mật độ xác suất**.

Khi đó, ta thu được **ước lượng mật độ nhân (Kernel Density Estimator – KDE)**, trong đó hàm chỉ báo được thay bằng **hàm nhân (kernel function)** như Gaussian, Epanechnikov, hay Uniform.

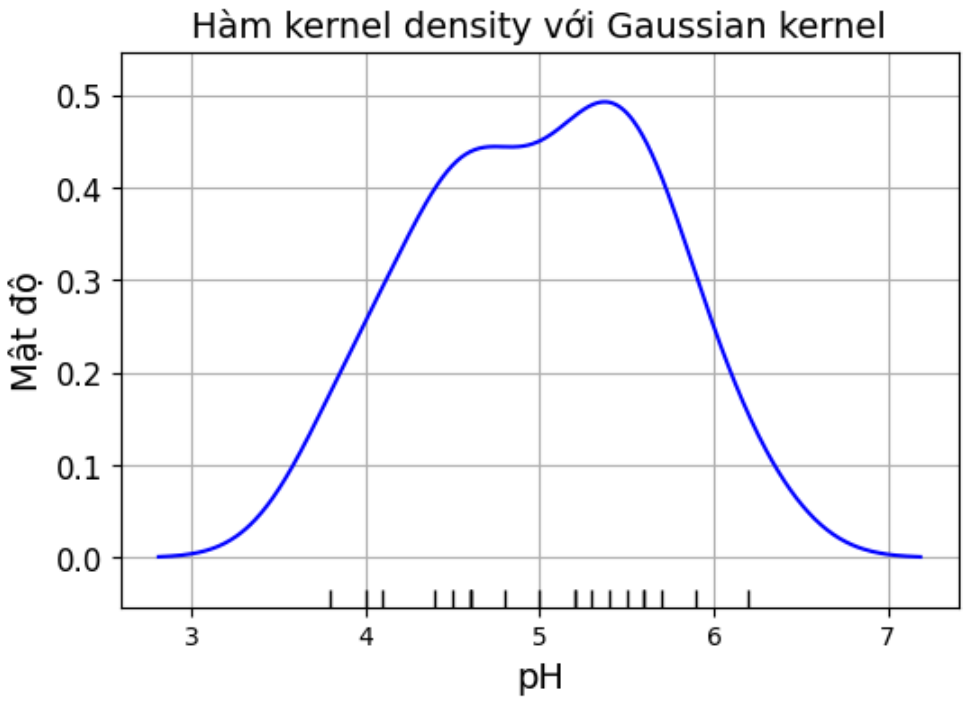
Công thức tổng quát của KDE được viết là:

$$\widehat{f}_{K,n}(y) = \frac{1}{nh} \sum_{i=1}^{n} k\left(\frac{Y_i - y}{h}\right).$$

Như vậy, KDE chính là một **phiên bản trơn hóa** của EDENS, giúp ước lượng mật độ xác suất trở nên **liên tục, mượt mà và dễ sử dụng hơn** trong phân tích dữ liệu.

In [70]:
```
from sklearn.neighbors import KernelDensity

# --- Tính bandwidth ---
def bw_nrd0(x):
    n = len(x)
    iqr = np.quantile(x, 0.75) - np.quantile(x, 0.25)  # IQR = Q3 - Q1
    std_x = x.std(ddof=1)
    return 0.9 * min(std_x, iqr / 1.34) * n ** (-1/5)

bw = bw_nrd0(catchments["pH"])

# --- Tạo mô hình KDE với kernel Gaussian ---
kde = KernelDensity(kernel='gaussian', bandwidth=bw)
kde.fit(catchments[["pH"]])

# --- Tạo lưới giá trị pH ---
x_min, x_max = catchments["pH"].min(), catchments["pH"].max()
x0 = np.linspace(x_min - 3*bw, x_max + 3*bw, 500).reshape(-1, 1)
log_dens = kde.score_samples(x0)
y = np.exp(log_dens)

# --- Vẽ biểu đồ ---
plt.figure(figsize=(6, 4))
plt.plot(x0, y, color="blue")

# Rug plot (các vạch nhỏ ở trục x)
sns.rugplot(
    x=catchments["pH"],
    height=0.03,
    color="black"
)

plt.xlabel("pH", fontsize=14)
plt.ylabel("Mật độ", fontsize=14)
plt.title("Hàm kernel density với Gaussian kernel", fontsize=14)

plt.yticks(fontsize=12)
plt.grid(True)

plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have v
alid feature names, but KernelDensity was fitted with feature names
  warnings.warn(
```



BT: Xét lại dữ liệu của 20 mẫu đất tại 20 lưu vực sông trong dữ liệu `catchments.csv` hãy biểu diễn mật độ dữ liệu được ước lượng bởi phương pháp kernel với Gaussian kernel cùng độ rộng tối ưu. Từ đó đưa ra nhận xét.

2.4.1 Hàm mật độ cho phép biến đổi log (log-transformation)

Các hàm mật độ ước lượng (Kernel Density Estimation – KDE) thông thường giả định rằng dữ liệu nằm trong miền thực. Tuy nhiên, nhiều biến thực tế không nằm trên toàn trục thực, ví dụ: Phần trăm, tỉ lệ; Nồng độ, tỉ lệ Ca/Mg → chỉ dương; Xác suất, tỷ lệ thực vật, độ ẩm, v.v.

Nếu ta áp dụng KDE trực tiếp trên những biến này, mật độ ước lượng sẽ "tràn" ra ngoài miền hợp lệ, ví dụ mật độ âm hoặc giá trị ở ngoài [0,1].

Đầu tiên, cần biến đổi log cho cột `CaMg` của dữ liệu. Dữ liệu sau khi biến đổi sẽ là một `DataFrame` với cột là `CaMg` đã được log đổi biến.

In [71]:
```
# --- Log-transform ---
x_camg = np.log(catchments[['CaMg']])
x_camg.head(5)
```

Out[71]:

	CaMg
0	-1.049822
1	2.397895
2	1.791759
3	0.587787
4	0.832909
5	-0.527633
6	2.128232
7	1.064711
8	1.774952
9	0.741937
10	0.182322
11	0.741937
12	0.095310
13	0.470004
14	-0.105361
15	0.530628
16	1.223775
17	-0.634878
18	0.788457
19	2.041220

Tính giá trị `bandwidth` theo công thức `Silverman` như đã đề cập ở phía trên. Giá trị trả về sẽ là độ rộng tối ưu cho biến `CaMg` đã được đổi biến log.

In [72]:
```
# --- Tính bandwidth theo Silverman ---
n_camg = len(x_camg)
iqr_camg = np.quantile(x_camg['CaMg'], 0.75) - np.quantile(x_camg['CaMg'], 0.25)
std_x_camg = np.std(x_camg['CaMg'], ddof=1)
bw_camg = 0.9 * min(std_x_camg, iqr_camg / 1.34) * n_camg ** (-1/5)

print('Độ rộng tối ưu: ', bw_camg)
```

```
Độ rộng tối ưu:  0.4430727721788258
```

Sau đó, gọi hàm `KernelDensity` cho biến `CaMg` đã biến đổi log với kernel Gaussian và độ rộng tối ưu đã tính ở phía trên.

In [73]:
```
# --- KDE với Gaussian kernel ---
kde = KernelDensity(kernel='gaussian', bandwidth=bw_camg)
# x_camg là cấu trúc DataFrame với biến CaMg đã được đổi biến log
kde.fit(x_camg[['CaMg']])
```

Out[73]:

```
KernelDensity
KernelDensity(bandwidth=np.float64(0.4430727721788258))
```

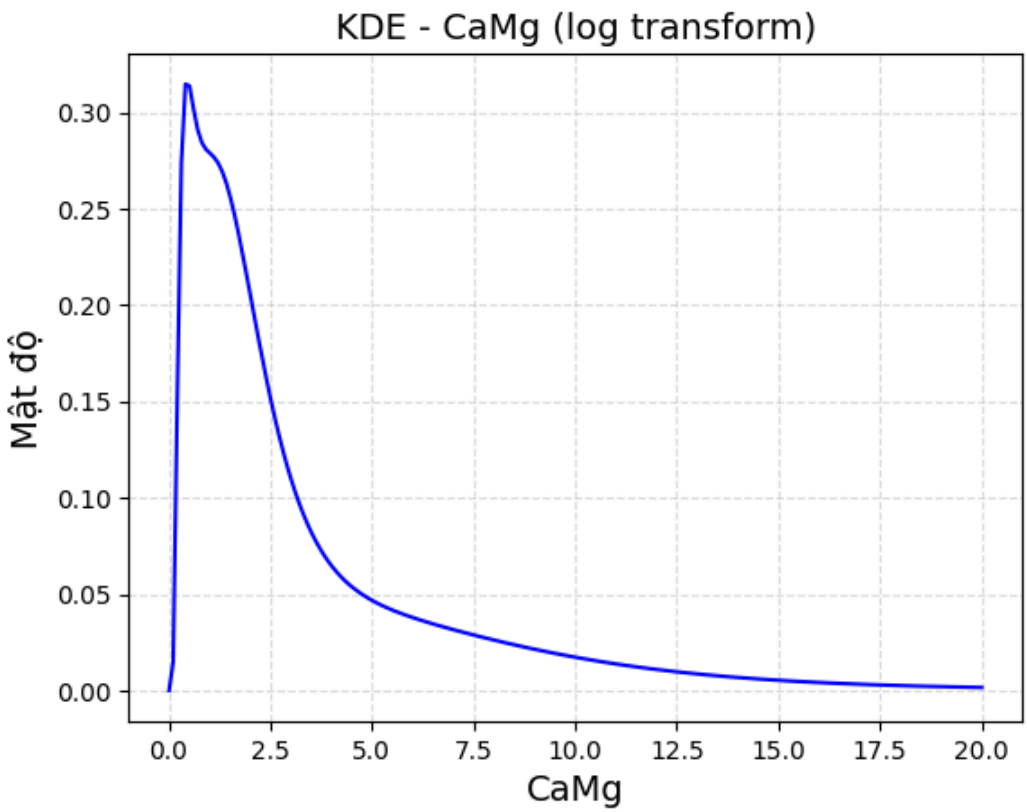
Sau khi biến đổi các biến theo các hàm log-transform, chúng ta cần đưa chúng lại thang gốc biểu đồ với trục x là CaMg thay vì giá trị log, từ đó đánh giá dữ liệu và trực quan hợp lý hơn.

Tạo các điểm x_grid để ước lượng mật độ xác suất Kernel cho x_grid đã được đổi biến log sau đó với giá trị ước lượng cần mũ giá trị ước lượng và nhân với `1/x_grid`

In [74]:
```
# --- Lưới giá trị CaMg ---
x_grid = np.linspace(0.01, 20, 200)
log_grid = np.log(x_grid).reshape(-1, 1)  # Đổi biến lưới x_grid về dạng log

# --- Ước lượng mật độ và hiệu chỉnh về thang gốc ---
log_dens = kde.score_samples(log_grid)
y = np.exp(log_dens) / x_grid  # đạo hàm log(x) = 1/x
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have v
alid feature names, but KernelDensity was fitted with feature names
  warnings.warn(
```

In [75]:
```
# --- Vẽ biểu đồ ---
plt.plot(x_grid, y, color="blue")
plt.xlabel("CaMg", fontsize=14)
plt.ylabel("Mật độ", fontsize=14)
plt.title("KDE - CaMg (log transform)", fontsize=14)
plt.grid(True, linestyle="--", alpha=0.4)
plt.show()
```



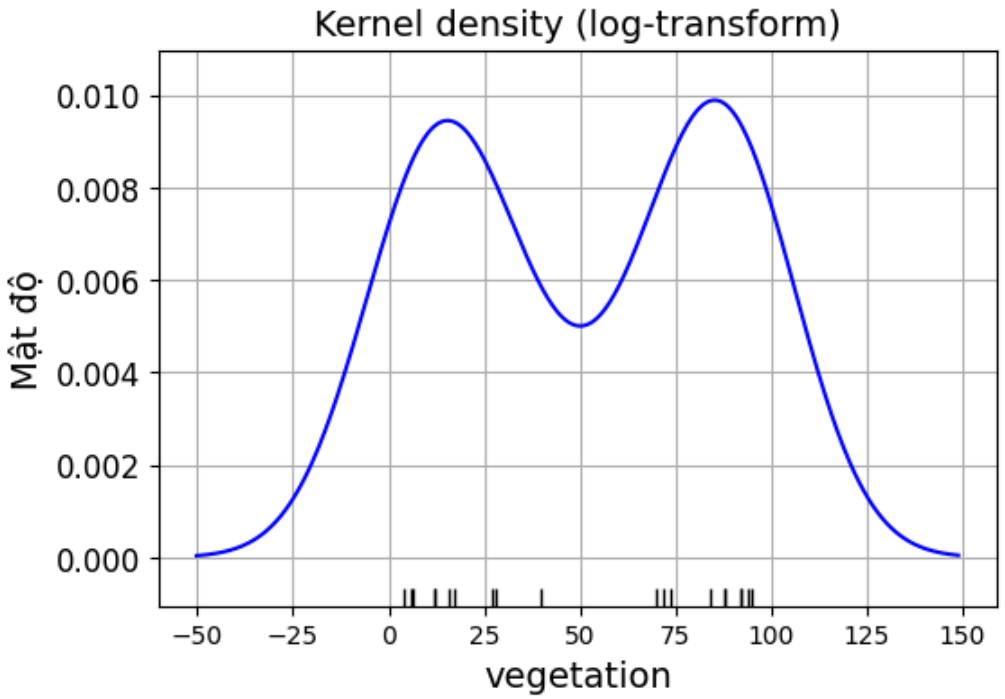
2.4.2 Hàm mật độ cho phép biến đổi logistic - (logistic transformation)

Một ví dụ khác với giá trị cột `vegetation` trong dữ liệu `catchments` ở phía trên với việc vẽ hàm mật độ xác suất qua hàm `KernelDensity` với Kernel = `Gaussian` và độ rộng tối ưu.

In [76]:
```
# --- Tính bandwidth ---
bw = bw_nrd0(catchments['vegetation'])

# --- Tạo mô hình KDE với kernel Gaussian ---
kde = KernelDensity(kernel='gaussian', bandwidth=bw)
kde.fit(catchments[['vegetation']])
```

```
# --- Tạo lưới giá trị pH ---
x_min, x_max = catchments['vegetation'].min(), catchments['vegetation'].max()
x0 = np.linspace(x_min - 3*bw, x_max + 3*bw, 500).reshape(-1, 1)
log_dens = kde.score_samples(x0)
y = np.exp(log_dens)

# --- Vẽ biểu đồ ---
plt.figure(figsize=(6, 4))
plt.plot(x0, y, color="blue")

# Rug plot
sns.rugplot(
    x=catchments['vegetation'],
    height=0.03,
    color="black"
)

plt.xlabel("vegetation", fontsize=14)
plt.ylabel("Mật độ", fontsize=14)
plt.title("Kernel density (log-transform)", fontsize=14)

plt.yticks(fontsize=12)
plt.grid(True)

plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have v
alid feature names, but KernelDensity was fitted with feature names
  warnings.warn(
```



Có thể thấy, với các giá trị của cột `vegetation` là dạng phần trăm, tuy nhiên ước lượng mật độ xác suất lại ước lượng cho các giá trị âm và lớn hơn 100. Điều đó cho thấy, việc ước lượng hiện tại là không hợp lý và cần biến đổi logistic để giá trị ước lượng hợp lý hơn

Đầu tiên, cần chuyển đổi các giá trị của cột `vegetation` về dạng (0, 1). Sau đó dùng phép đổi biến logit

In [77]:
```
# --- Chuẩn hóa vegetation về [0,1] ---
veg = catchments[['vegetation']] / 100

# Dùng phép đổi biến logit cho biến veg
x_veg = np.log(veg / (1 - veg))
```

```
x_veg.head(5)
```

Out[77]:

	vegetation
0	-2.787562
1	-0.944462
2	-1.992430
3	-0.994623
4	-0.405465

Tiếp theo, tính giá trị độ rộng tối ưu với các giá trị đã được đổi biến logit.

In [87]:

```
# --- Tính bandwidth theo Silverman ---
n_x_veg = len(x_veg)
iqr_x_veg = np.quantile(x_veg['vegetation'], 0.75) - np.quantile(x_veg['vegetation'], 0.25)
std_x_veg = np.std(x_veg['vegetation'], ddof=1)
bw_x_veg = 0.9 * min(std_x_veg, iqr_x_veg / 1.34) * n_x_veg ** (-1/5)

print('Độ lệch chuẩn: ',std_x_veg)
print('Độ rộng tối ưu: ', bw_x_veg)
```

```
Độ lệch chuẩn:  2.081039485453945
Độ rộng tối ưu:  1.0287665405017963
```

Gọi hàm `KernelDensity` với các giá trị đã đổi biến logit và độ rộng tối ưu như đã tính ở phía trên.

In [88]:

```
from sklearn.neighbors import KernelDensity

# --- KDE với Gaussian kernel ---
kde = KernelDensity(kernel='gaussian', bandwidth=bw_x_veg)
kde.fit(x_veg[['vegetation']])
```

Out[88]:

```
KernelDensity
KernelDensity(bandwidth=1.0287665405017963)
```

Tạo lưới giá trị x là biến đổi logit với lưới giá trị, sau đó ước lượng và hiệu chỉnh về thang gốc bằng cách `/ x_grid * (1 - x_grid)`

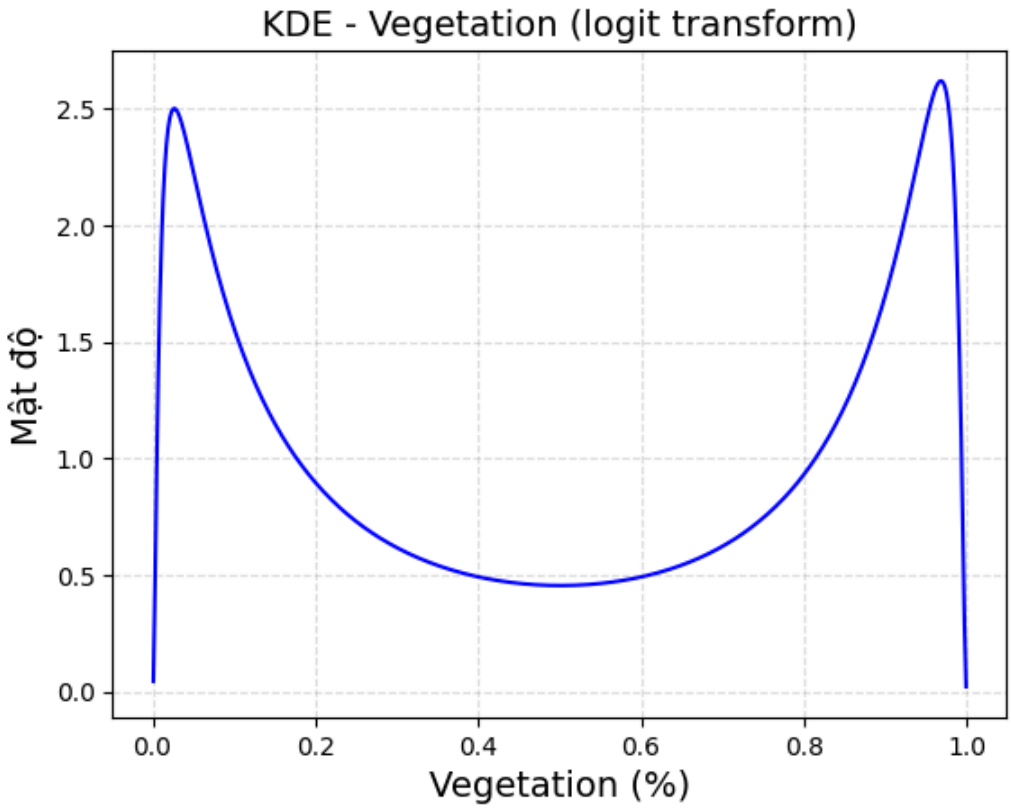
In [80]:

```
# --- Lưới giá trị vegetation (0-100%) ---
x_grid = np.linspace(0.001, 0.999, 500)
logit_grid = np.log(x_grid / (1 - x_grid)).reshape(-1, 1)

# --- Ước lượng mật độ và hiệu chỉnh về thang gốc ---
log_dens = kde.score_samples(logit_grid)
y = np.exp(log_dens) / (x_grid * (1 - x_grid))  # hiệu chỉnh đạo hàm logit
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have v
alid feature names, but KernelDensity was fitted with feature names
  warnings.warn(
```

In [81]:

```
# --- Vẽ biểu đồ ---
plt.plot(x_grid, y, color="blue")
plt.xlabel("Vegetation (%)", fontsize=14)
plt.ylabel("Mật độ", fontsize=14)
plt.title("KDE - Vegetation (logit transform)", fontsize=14)
plt.grid(True, linestyle="--", alpha=0.4)
plt.show()
```



2.5 Biểu đồ tam phân

In [82]:
```
!pip install python-ternary   # Tải thư viện về
```

```
Collecting python-ternary
  Downloading python_ternary-1.0.8-py3-none-any.whl.metadata (1.0 kB)
Requirement already satisfied: matplotlib>=2 in /usr/local/lib/python3.12/dist-packages (from python-ter
nary) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplot
lib>=2->python-ternary) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>
=2->python-ternary) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplo
tlib>=2->python-ternary) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplo
tlib>=2->python-ternary) (1.4.9)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=
2->python-ternary) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotl
ib>=2->python-ternary) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2-
>python-ternary) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplot
lib>=2->python-ternary) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from mat
plotlib>=2->python-ternary) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil
>=2.7->matplotlib>=2->python-ternary) (1.17.0)
Downloading python_ternary-1.0.8-py3-none-any.whl (18 kB)
Installing collected packages: python-ternary
Successfully installed python-ternary-1.0.8
```

In [83]:
```
df = pd.read_csv('/content/drive/MyDrive/course_Data_Science_for_Geoscience/Week 1/Dataset/ACNK.csv')
```

In [84]:
```
df.head(5)
```

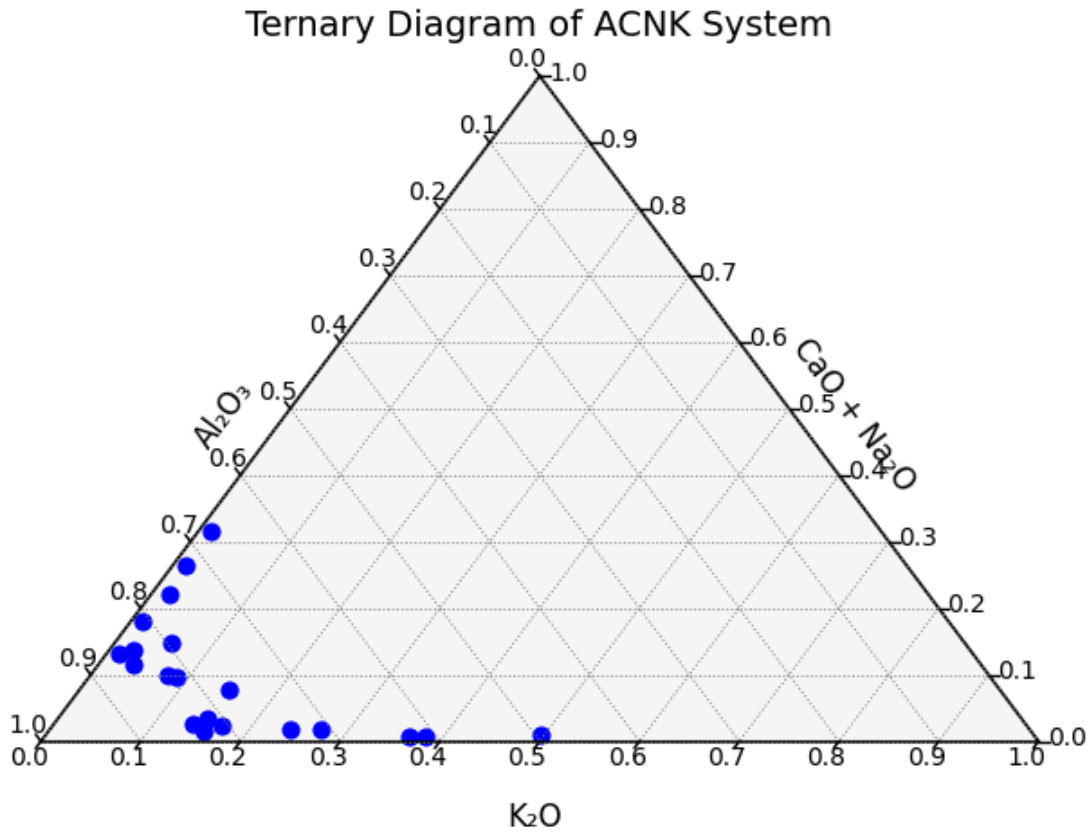
Out[84]:

	Al2O3	CaO+Na2O	K20
0	0.796	0.057	0.147
1	0.494	0.496	0.010
2	0.627	0.366	0.007
3	0.838	0.024	0.138
4	0.711	0.271	0.018

In [85]:

```
import pandas as pd
import matplotlib.pyplot as plt
import ternary

# --- Tạo biểu đồ tam giác ---
fig, tax = ternary.figure()

# --- Vẽ điểm ---
points = df[["CaO+Na2O", "K20", "Al2O3"]].values.tolist()
tax.scatter(points, color="blue", s=40)

# --- Vẽ khung và lưới ---
tax.boundary(linewidth=1.2)
tax.gridlines(color="gray", multiple=0.1, linewidth=0.7)
tax.ticks(axis='lbr', multiple=0.1, linewidth=1, tick_formats="%.1f")
tax.clear_matplotlib_ticks()

# --- Nhãn trục ---
tax.left_axis_label("Al₂O₃", fontsize=12)
tax.right_axis_label("CaO + Na₂O", fontsize=12)
tax.bottom_axis_label("K₂O", fontsize=12)

# --- Xóa khung ngoài và thêm tiêu đề ---
for spine in tax.ax.spines.values():
    spine.set_visible(False)

tax.set_title("Ternary Diagram of ACNK System", fontsize=14)
plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/ternary/plotting.py:148: UserWarning: No data for colormapping p
rovided via 'c'. Parameters 'vmin', 'vmax' will be ignored
  ax.scatter(xs, ys, vmin=vmin, vmax=vmax, **kwargs)
```


Ternary Diagram of ACNK System
Al2O3
CaO + Na2O
K2O
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0